Lab 6

姓名	學號
楊正豪	314553044

---

1. Introduction

What is DDPM?

DDPM (Denoising Diffusion Probabilistic Models) 是一種生成模型。它的做法是：先把一張真實圖片逐步加入雜訊，多步之後讓它變成一片純雜訊，接著訓練一個神經網路學習反過來的事，給它一張加了雜訊的圖，讓它預測加進去的那個噪音是什麼。

Task Overview

本次 Lab 實作 Conditional DDPM，使模型能根據給定的多個條件生成對應的影像。實驗以 iCLEVR 資料集為基準，藉由性能評估，驗證 DDPM 模型在複雜場景下精準生成多物件特徵的能力與表現。

---

2. Implementation Details

2.1 Noise Scheduler

使用 HuggingFace Diffusers 提供的 `DDPMScheduler`，搭配 Cosine Noise Scheduler `squaredcos_cap_v2`。

Squared Cosine Schedule

傳統的 Linear Schedule 會讓雜訊在圖片上累積的速度太快，導致後期圖片被破壞得過於徹底，模型無法在那個階段學到有用的資訊。因此我選擇 `squaredcos_cap_v2` 作為 Noise Schedule，加噪節奏是一開始慢、中間快、後期接近純噪音時又慢下來，使模型能較均衡地學習每個階段的去噪任務。

各符號定義：

- β_t：每一步加入的 Noise Rate
- $\alpha_t = 1 - \beta_t$：每一步保留的比例
- $\bar{\alpha}_t = \frac{f(t)}{f(0)}$：在第 t 步時，原始圖片還剩下多少比例

Add Noise

x_0 在訓練階段依照以下公式產生出加噪後的圖像 x_t：

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$$

Denoise

在 Inference 階段，從 x_t 利用訓練好的 UNet，一步一步利用以下公式去噪，直到還原 x_0：

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \epsilon_\theta(x_t, t)\right) + \sigma_t \cdot z$$

```
# train.py
noise_scheduler = DDPMScheduler(
    num_train_timesteps=args.timesteps,
    beta_schedule="squaredcos_cap_v2"
)

# test.py
noise_scheduler = DDPMScheduler(
    num_train_timesteps=500,
    beta_schedule="squaredcos_cap_v2"
)
noise_scheduler.set_timesteps(500)
```

2.2 Model Architecture

整個 Diffusion Model 分為兩個部分：

1. 將條件嵌入的 Condition Embedding
2. 使用 HuggingFace Diffusers 函式庫內的 `UNet2DModel` 作為核心網路

2.2.1 Condition Embedding

若將 1×24 的條件標籤與 64×64 的影像直接進行通道拼接，會導致 CNN 難以提取有效的紋理與邊緣特徵。因此本實驗採用投影機制，先透過 Linear Layer 將條件標籤轉換為 512 維的高維嵌入向量，並將其與 Time Step 向量相加融合。此綜合引導向量隨後被注入 U-Net 的各個模組中，以實現精準的條件影像生成。

```
# unet.py
NUM_OBJECTS = 24
COND_DIM    = 512

class ConditionEmbedding(nn.Module):
    def __init__(self, num_objects=NUM_OBJECTS, cond_dim=COND_DIM):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(num_objects, cond_dim),
            nn.SiLU(),
            nn.Linear(cond_dim, cond_dim),
            nn.SiLU(),
        )

    def forward(self, label):
        return self.net(label)
```

2.2.2 UNet

去噪的核心是一個 U 形的卷積網路 UNet，架構如下：

```
# unet.py
self.unet = UNet2DModel(
    sample_size        = 64,
    in_channels        = 3,
    out_channels       = 3,
    class_embed_type   = "identity",
    layers_per_block   = 2,
    block_out_channels = (128, 256, 256, 512),
    down_block_types   = (
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "AttnDownBlock2D",
    ),
    up_block_types = (
        "AttnUpBlock2D",
        "AttnUpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
    ),
)
```

Encoder（下採樣，壓縮圖片）：

區塊	類型	尺寸變化	通道數
Down 1	`DownBlock2D`	64 → 32	128
Down 2	`DownBlock2D`	32 → 16	256
Down 3	`AttnDownBlock2D`	16 → 8	256
Down 4	`AttnDownBlock2D`	8 → 4	512

Decoder（上採樣，還原圖片）：

區塊	類型	尺寸變化	通道數
Up 1	`AttnUpBlock2D`	4 → 8	512
Up 2	`AttnUpBlock2D`	8 → 16	256
Up 3	`UpBlock2D`	16 → 32	256
Up 4	`UpBlock2D`	32 → 64	128

2.2.3 Condition 與 Time Embedding 的注入

將 Time Step 與 Condition 進行整合。具體做法是利用 2.2.1 所述將標籤轉換為 512 維的高維向量，並與時間步向量進行相加融合。此設計旨在使 U-Net 的各個模組均能接收到一致的資訊，進而

在去噪過程的各個階段精準實現影像內容的條件化特徵調節。

```
# unet.py
def forward(self, x, timestep, label):
    cond = self.cond_emb(label)
    return self.unet(x, timestep, class_labels=cond).sample
```

2.3 Dataloader

在 `data.py` 中實作自訂的 `Object` Dataset，支援三種模式：

Mode	資料來源	回傳內容	資料量
`train`	`train.json` ＋ `iclevr/` 圖片	圖片＋標籤向量	18,009
`test`	`test.json`	標籤向量	32
`new_test`	`new_test.json`	標籤向量	32

`train` 模式會載入圖片檔名與對應標籤，從磁碟讀取真實圖片；`test` / `new_test` 模式只提供條件標籤，由模型負責從純噪音生成對應圖片。

標籤處理

將每個物件轉換成 one-hot 向量，長度為 24 的 0/1 向量：

```
# data.py
label = [0] * 24
for obj_name in self.labels[img_filename]:
    label[self.objects[obj_name]] = 1
label = torch.tensor(label, dtype=torch.float32)
```

圖片 Preprocessing

Input 圖片 Resize 縮放到 64×64，並正規化到 -1 至 1 的範圍。

```
# data.py
self.transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                         std =[0.5, 0.5, 0.5])
])
```

DataLoader 設定

```
# train.py
train_loader = DataLoader(
    train_dataset,
    batch_size  = args.batch_size,
```

```
    shuffle     = True,
    num_workers = args.num_workers,
    pin_memory  = True
)
```

2.4 Training Details

Optimizer and Learning Rate Schedule

```
# train.py
optimizer = torch.optim.AdamW(
    model.parameters(), lr=args.lr, weight_decay=1e-4
)

total_training_steps = len(train_loader) * args.num_epochs
lr_scheduler = get_cosine_schedule_with_warmup(
    optimizer          = optimizer,
    num_warmup_steps   = args.lr_warmup_steps,
    num_training_steps = total_training_steps
)
```

Setting	Value
Optimizer	AdamW
Learning Rate	0.0001
Weight Decay	0.0001
LR Schedule	Cosine Decay + Linear Warmup
Warmup Steps	500
Total Epochs	200
Batch Size	64

使用 AdamW 能更穩定地防止權重過大導致的過擬合。

Warmup 讓學習率從 0 緩慢爬升，給模型一段熱身期先穩住方向，再正式加速。後期 Cosine Decay 讓學習率緩慢下降，在 Loss 快收斂時還能繼續精細調整。

Loss Function

我採用 **MSE (Mean Squared Error)** 作為 Loss Function，訓練 UNet 預測加噪時加入的高斯噪音 ϵ，而非直接預測原始圖片 x_0。

```
# train.py
pred_noise = model(noisy_images, t, labels)
loss = F.mse_loss(pred_noise, noise)
```

Training Loop

每個 batch 的訓練步驟依序是：

1. 隨機將部分標籤歸零作為無條件訓練信號（CFG Dropout，詳見 3.5）
2. 對圖片加入對應時間步程度的高斯噪音
3. 讓 UNet 看加噪圖、時間步、條件標籤，預測加入的噪音
4. 計算預測噪音和真實噪音的 MSE
5. 反向傳播、裁剪梯度、更新參數與學習率
6. 記錄當前 Loss 與學習率至 WandB

```
# train.py
for images, labels in train_loader:
    images = images.to(device)
    labels = labels.to(device).float()
    bs = images.shape[0]

    if args.cfg_drop_rate > 0:
        drop_mask = torch.rand(bs, 1, device=device) < args.cfg_drop_rate
        labels = torch.where(drop_mask, torch.zeros_like(labels), labels)

    noise = torch.randn_like(images)
    t = torch.randint(0, args.timesteps, (bs,), device=device).long()
    noisy_images = noise_scheduler.add_noise(images, noise, t)
    pred_noise = model(noisy_images, t, labels)
    loss = F.mse_loss(pred_noise, noise)

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    lr_scheduler.step()
```

Checkpoint Strategy

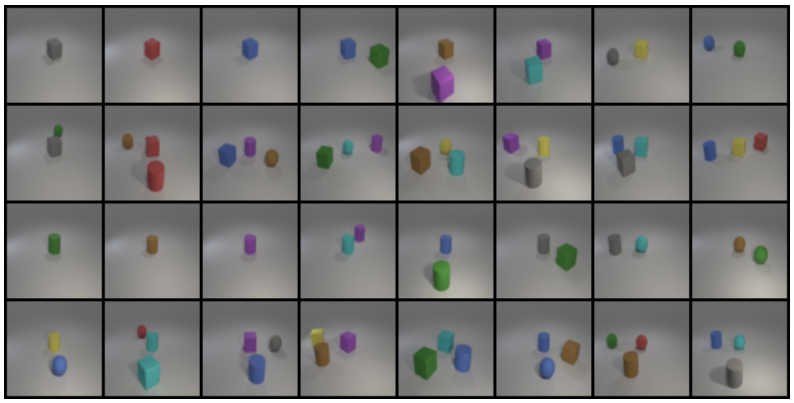
---

3. Results and Discussion

3.1 Synthetic Image Grids

以下為模型針對 `test.json` 與 `new_test.json` 兩組條件所生成的圖像。

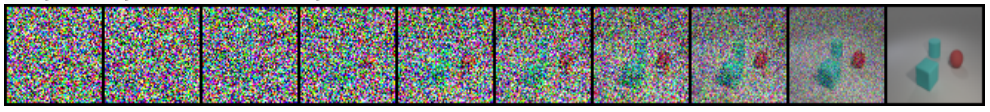
Test Set



New Test Set

3.2 Denoising Process

以下展示從純高斯噪音 x_T 逐步去噪還原至最終圖像 x_0 的過程，使用條件標籤 `["red sphere", "cyan cylinder", "cyan cube"]`



3.3 Experimental Results

以下為使用 cfg_scale = 3.0、權重 `ddpm_epoch_200.pth` 對兩組測試集進行評分的結果。

```
 Mode: test
Denoising: 100%|
test Accuracy: 0.8889

 Mode: new_test
Denoising: 100%|
new_test Accuracy: 0.9048
Generating denoising process: 100%|
Save path:results\denoising process.png
```

Test Set	Accuracy
test	0.88
new_test	0.90

3.5 Extra Experiment — Classifier-Free Guidance

CFG (Classifier-Free Guidance) 是一個讓條件控制更強的技巧，以下為 CFG 的公式：

$$\epsilon_{final} = \epsilon_{uncond} + w \cdot (\epsilon_{cond} - \epsilon_{uncond})$$

透過控制 w (CFG Scale)，迫使模型能夠遵循 condition 的約束，但如果 CFG Scale 過高，會導致雜訊偏離合理的圖片分布，以下為實作細節：

Training：

在訓練過程中，以 cfg_drop_rate 機率隨機將標籤向量替換為沒有條件的狀態，讓模型同時學習兩件事：有條件時要怎麼去噪，以及完全沒有條件時要怎麼去噪。這樣使得在遇到 OOD 的情況，模型也能夠精準地去噪。

```
# train.py
if args.cfg_drop_rate > 0:
    drop_mask = torch.rand(bs, 1, device=device) < args.cfg_drop_rate
    labels = torch.where(drop_mask, torch.zeros_like(labels), labels)
```

Testing：

推理時，對同一張噪音圖，模型同時做兩次預測：一次餵入有標籤，一次餵入無標籤。兩者被合併成一個 batch 一次完成 forward。

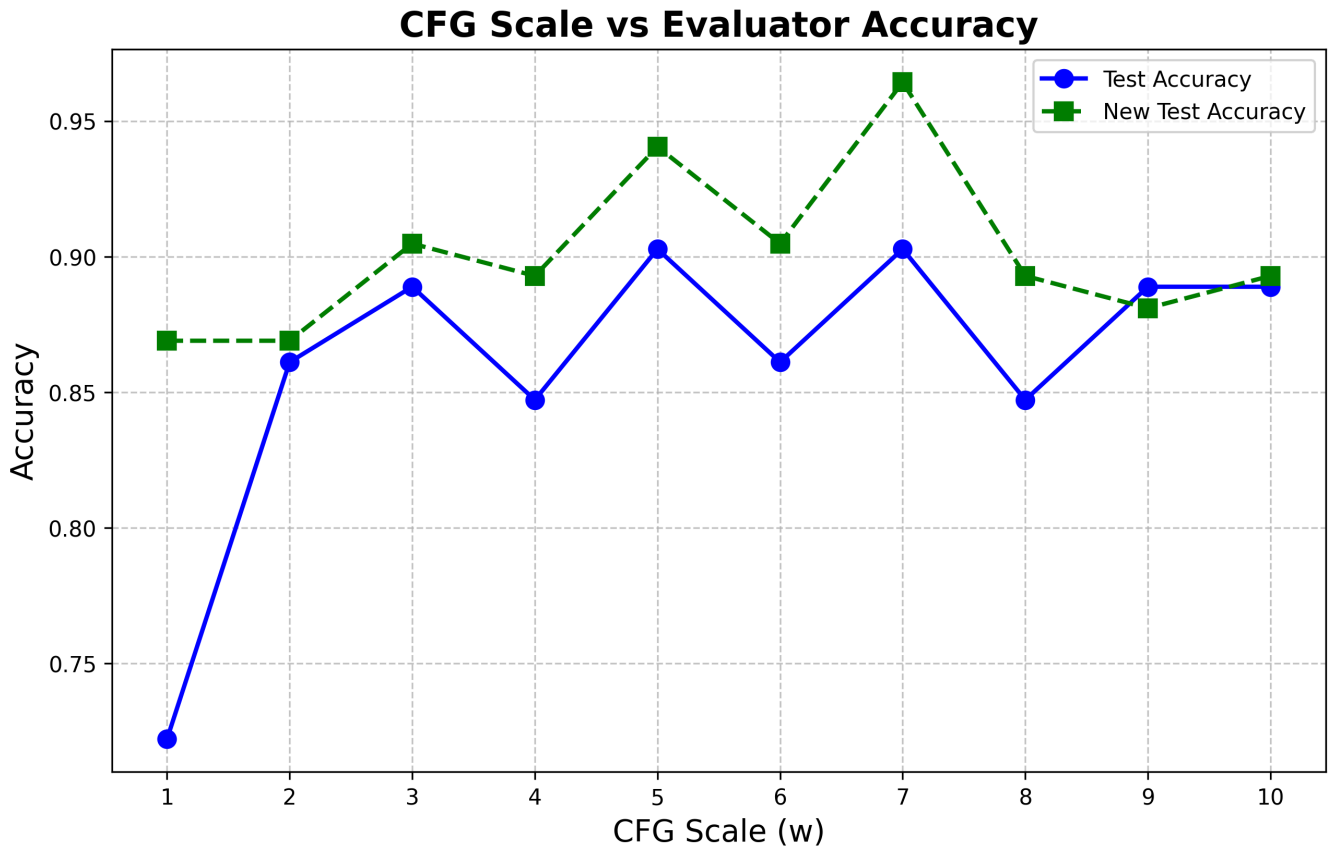
```
# test.py
for t in tqdm(noise_scheduler.timesteps, desc="Denoising"):
    latent_model_input = torch.cat([current_images, current_images], dim=0)
    combined_labels    = torch.cat([labels, uncond_labels], dim=0)

    t_val = t.item() if isinstance(t, torch.Tensor) else t
    t_for_model = torch.full((latent_model_input.shape[0],), t_val,
                             device=device, dtype=torch.long)

    noise_pred = model(latent_model_input, t_for_model, combined_labels)

    noise_pred_cond, noise_pred_uncond = noise_pred.chunk(2, dim=0)
    noise_pred = noise_pred_uncond + args.cfg_scale * (noise_pred_cond -
noise_pred_uncond)

    current_images = noise_scheduler.step(noise_pred, t,
current_images).prev_sample
```



---

4. References

HuggingFace Diffusers — https://github.com/huggingface/diffusers

提供擴散模型訓練與推理所需的核心元件，本實驗使用以下三個功能：

Function	用途
`DDPMScheduler`	加噪與反向去噪的時間步計算，支援 Cosine Noise Schedule
`UNet2DModel`	提供帶有 Self-Attention 的 U-Net 骨幹網路，作為去噪模型的主體
`get_cosine_schedule_with_warmup`	生成含 Linear Warmup 的 Cosine 學習率排程器